Topic: Terraform Backend — Local vs Remote State, S3 + DynamoDB State Locking, Versioning, Migrating State

(Your notes, cleaned up from dictation + screenshots — Lecture 8)

Quick recall

> **Definition:** A **backend defines where Terraform stores its state data files.**
>
> The primary function of a backend is to **store the state created by Terraform runs** after provisioning our resources. This gives Terraform **a single place to look up what the expectation of our resources is from the last time we applied.**

This is a very important topic. Chapter 02 covered *what* `terraform.tfstate` is. This chapter is about *where* it lives, and why "on my laptop" isn't good enough for a team.

---

1. Two ways to save the Terraform state file

1. Saving locally (local backend)

A local backend stores the state file **on the machine where Terraform is running**. This is the **default backend**, used if you don't specify a backend in your Terraform configuration.

The local backend is **useful for testing and development environments, but it's not recommended for production**. It can lead to inconsistencies if the state file is lost or corrupted.

2. Saving remotely (remote backend)

A remote backend stores the state file in a **centralized location**, such as a cloud object storage service or a database. Remote backends give several benefits:

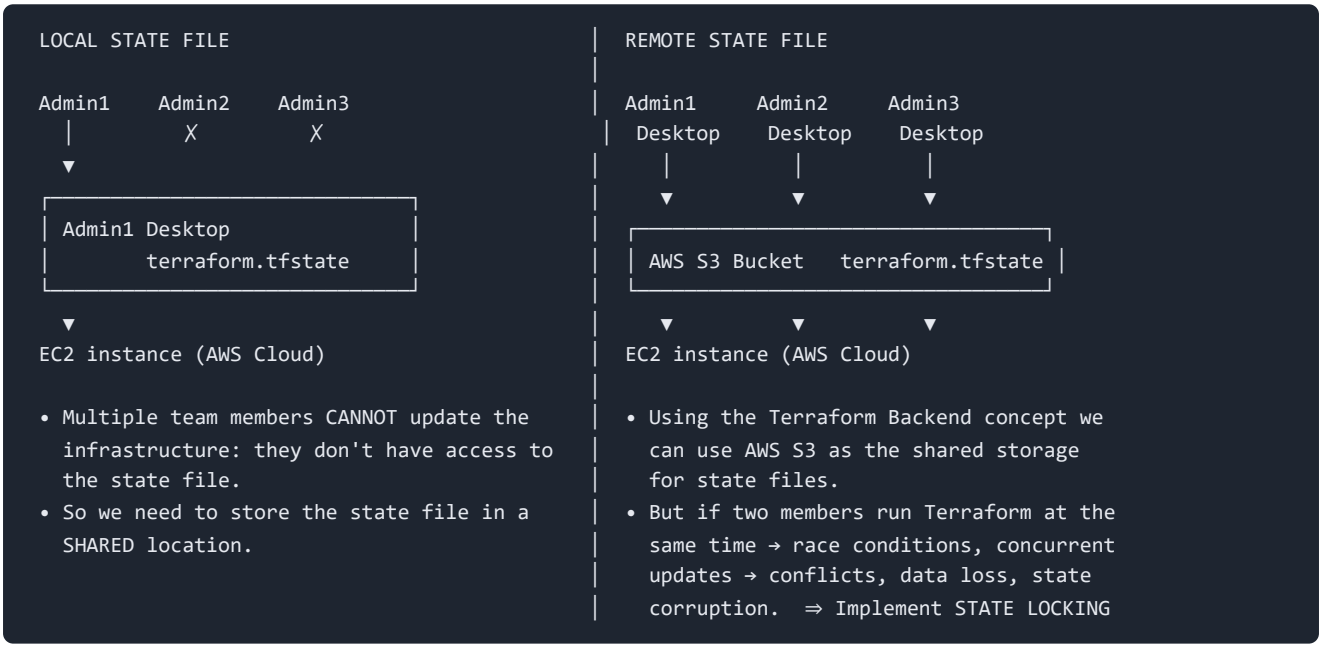
- they enable **collaboration** between team members
- they allow **versioning** of state files
- they provide a **history of changes**

There are several remote backend providers available, such as **Amazon S3, Azure Storage (`azurerm`), Google Cloud Storage (`gcs`)**, HCP Terraform / Terraform Cloud, Consul, etc.

Advantages of using a remote backend

1. **Centralized storage** of the state file, which enables collaboration between team members working on the same project.
2. **Versioning of state files**, which lets you manage and track changes to your infrastructure over time.
3. **History of changes**, which helps with troubleshooting and auditing.
4. Features like remote **state locking** prevent conflicts and ensure consistency. *(One of the best and most important features you get with a remote backend.)*

The picture: local state file vs remote state file



---

2. What is locking and why do we need it?

If the state file is stored remotely so that many people can access it, you risk **multiple people attempting to make changes to the same file at the exact same time**. Simultaneous operations could lead to conflicting changes, **potentially corrupting the state**.

Example: suppose you have a team of 10 DevOps engineers working on different tasks. They're making changes against the **same state file at the exact same time**. That simultaneous operation can lead to conflicting changes, and the state file can get corrupted.

So we need a mechanism that **"locks" the state if it's currently in use by another user**. If one DevOps engineer is running against the state file, it's locked, and everyone else has to wait. In this lab, Terraform state locking is provided by **AWS S3 + DynamoDB**.

> 📝 **Update for Terraform ≥ 1.10/1.11:** the S3 backend can now lock **natively in S3** with `use_lockfile = true` (it writes a `<key>.tflock` object). `dynamodb_table` **is deprecated** and will be removed in a future version. DynamoDB locking still works (the lab used v1.13.3), but for new setups prefer `use_lockfile`. See §8.

---

3. Terraform backend configuration

The Terraform backend is configured **like any other configuration in the configuration file**, and **when you run `terraform init`, the backend is initialized.**

Example: AWS S3 as a Terraform backend with DynamoDB locking:

```
terraform {
  backend "s3" {
    bucket         = "angelo-terraform-state-backend"
    key            = "terraform.tfstate"
    region         = "eu-west-1"
    dynamodb_table = "terraform_state"
  }
}
```

"This is the code that creates the backend. But is it so easy?" No. The **bucket and the table must already exist** before `terraform init`. The backend block doesn't create them. That's why the lab builds them first.

Recommended production setup

```
Terraform
   ↓
S3 Bucket        (state storage,  versioning ON)
   ↓
DynamoDB Table   (state locking)       ← or S3 native lock: use_lockfile = true
```

Benefits: safe collaboration, prevents simultaneous updates, versioning support.

Why locking matters. State locking prevents situations where:

- two engineers run `terraform apply` simultaneously
- one process overwrites another's state
- infrastructure drift or corruption occurs

---

4. Lab part 1: create the S3 bucket and DynamoDB table (still local state)

The lab runs in **Asia Pacific (Mumbai), ap-south-1**, using three resources: an **S3 bucket** (state storage), a **DynamoDB table** (locking), and later an **EC2 instance** to test with.

S3 bucket with versioning

```
resource "aws_s3_bucket" "example" {
  bucket        = "ramprateek"
  force_destroy = true
  tags = {
    Name = "Terraform State File Storage"
  }
}

resource "aws_s3_bucket_versioning" "versioning_example" {
  bucket = aws_s3_bucket.example.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

- `force_destroy = true` : by default it's `false`. With `true`, if we destroy the bucket, **everything under it is destroyed too** (all objects and versions). We set it on purpose for the lab. ⚠️ **Never** do this on a real state

bucket. In production use `lifecycle { prevent_destroy = true }` instead.

- Versioning is its **own resource**, `aws_s3_bucket_versioning`. It points at the bucket with `aws_s3_bucket.example.id`. (Since AWS provider v4, bucket sub-settings are separate resources.)

DynamoDB table for locking

```
resource "aws_dynamodb_table" "example_table" {
  name         = "terraform-state-locking"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"          ### primary key must be "LockID"

  attribute {
    name = "LockID"
    type = "S"                    # S for String
  }

  tags = {
    Environment = "Development"
    Project     = "Example"
  }
}
```

- `hash_key` **(partition key) must be exactly** `LockID` (case-sensitive, type String). Terraform writes its lock items using that key name.
- `PAY_PER_REQUEST`: no capacity to plan. Lock traffic is tiny, so it costs next to nothing.

```
[root@Devops-server project1]# terraform state list
aws_dynamodb_table.example_table
aws_s3_bucket.example
aws_s3_bucket_versioning.versioning_example

[root@Devops-server project1]# ll
-rw-r--r--. 1 root root  643 Jun 29 09:08 main.tf
-rw-r--r--. 1 root root 6540 Jun 29 09:09 terraform.tfstate         ← state is still LOCAL
-rw-r--r--. 1 root root 3970 Jun 29 09:09 terraform.tfstate.backup
```

In the console, the DynamoDB table `terraform-state-locking` has **0 items** (just the `LockID (String)` key), and the bucket `ramprateek` **has 0 objects**.

To create the backend we need: ✅ the S3 bucket, ✅ the DynamoDB table, ✅ **versioning enabled on the bucket**. All ready. Till now our state file is **present locally**. We don't want to keep it locally; we want it in S3.

---

5. Lab part 2: configure the backend and migrate local → S3

```
# back.tf
terraform {
  backend "s3" {
    bucket         = "ramprateek"
    key            = "state/terraform.tfstate"
    dynamodb_table = "terraform-state-locking"
  }
}
```

What it says: the backend is **S3**, the bucket is **ramprateek**, and the **key** means: create a folder named `state` in the bucket and keep my `terraform.tfstate` under it. Use the DynamoDB table `terraform-state-locking` for locking.

> 📝 There's no `region` in this block. The S3 backend **doesn't** take the region from the `provider "aws"` block. It worked here because the region came from the environment / AWS CLI config (`AWS_REGION` or `~/.aws/config`). In real code, always write `region = "ap-south-1"` (and `encrypt = true`) in the backend block.

Step 1: `terraform plan` → error

As soon as we run `terraform plan` , it errors, because we're moving from **local to a backend**:

```
Error: Backend initialization required, please run "terraform init"

Reason: Initial configuration of the requested backend "s3"

The "backend" is the interface that Terraform uses to store state,
perform operations, etc. ...
Changes to backend configurations require reinitialization. This allows
Terraform to set up the new configuration, copy existing state, etc. Please run
"terraform init" with either the "-reconfigure" or "-migrate-state" flags to
use the current configuration.
```

Step 2: `terraform init` → copy the existing state?

```
[root@Devops-server project1]# terraform init
Initializing the backend...
Do you want to copy existing state to the new backend?
  Pre-existing state was found while migrating the previous "local" backend to the
  newly configured "s3" backend. No existing state was found in the newly
  configured "s3" backend. Do you want to copy this state to the new "s3"
  backend? Enter "yes" to copy and "no" to start with an empty state.

  Enter a value: yes

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
```

> ⚠️ Answer `yes` . `no` starts with an **empty** state. Terraform would "forget" the bucket and table it already

> manages and try to create them again on the next apply.

Step 3: verify

- **S3:** `ramprateek / state/terraform.tfstate`, **6.4 KB**, last modified 09:14:27. The local 6540-byte file is now in S3.
- **DynamoDB:** 1 item, `ramprateek/state/terraform.tfstate-md5` with a **Digest** (`8f1aa7fc12fea4f22f70d77c317603e3`). This isn't the lock. It's the **checksum of the state file**, which Terraform uses to detect it read a stale copy from S3.

---

6. Lab part 3: test it with an EC2 instance and watch the lock

Now we create one simple resource to check that the backend works:

```
# ec2.tf
resource "aws_instance" "linux" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  tags = {
    Name   = "Test_server"
    source = "terraform"
  }
}
```

How locking works: while `terraform apply` is running

Run `terraform apply` and leave it waiting at the approval prompt. At the backend, the DynamoDB table now has **2 items**:

LockID	Digest	Info
`ramprateek/state/terraform.tfstate-md5`	`8f1aa7fc12…`	
`ramprateek/state/terraform.tfstate`		`{"ID":"f740b3e6-8099-8d48-b86e-de933a3ba167","Operation":"OperationTypeApply",…}`

Open the lock item and the **Info** shows exactly who holds it:

```
{"ID":"f740b3e6-8099-8d48-b86e-de933a3ba167",
 "Operation":"OperationTypeApply",
 "Info":"",
 "Who":"root@Devops-server",
 "Version":"1.13.3",
 "Created":"2026-06-29T03:46:32.211807515Z",
 "Path":"ramprateek/state/terraform.tfstate"}
```

It shows an **apply is going on**, and **who** is using the state file: user `root` on host `Devops-server` (confirmed with `whoami` → `root`, `hostname` → `Devops-server`).

Somebody else tries at the same time → error

Exactly at this time, if somebody else (here, a second terminal) runs `terraform plan` or `apply`, they get an error, because the state file is locked:

```
[root@Devops-server project1]# terraform plan

Error: Error acquiring the state lock

Error message: operation error DynamoDB: PutItem, https response error StatusCode: 400, RequestID:
8J05A4DDPEHILD27RHAK6HAJDNVV4KQNSO5AEMVJF66Q9ASUAAJG, ConditionalCheckFailedException: The conditional reque
Lock Info:
  ID:        f740b3e6-8099-8d48-b86e-de933a3ba167
  Path:      ramprateek/state/terraform.tfstate
  Operation: OperationTypeApply
  Who:       root@Devops-server
  Version:   1.13.3
  Created:   2026-06-29 03:46:32.211807515 +0000 UTC
  Info:

Terraform acquires a state lock to protect the state from being written
by multiple users at the same time. Please resolve the issue above and try
again. For most commands, you can disable locking with the "-lock=false"
flag, but this is not recommended.
```

How it works under the hood: acquiring the lock is a DynamoDB **conditional** `PutItem` ("create this LockID only if it doesn't exist"). The item already exists, so the condition fails (`ConditionalCheckFailedException`) and Terraform refuses to continue.

After apply completes → lock released

Once the root user on the DevOps machine **completes the apply**, the lock is **released**. The DynamoDB table is back to **one item** (only the `-md5` digest, which is updated with the new checksum).

State now lives in S3, not locally

```
[root@Devops-server project1]# ll
-rw-r--r--. 1 root root  139 Jun 29 09:11 back.tf
-rw-r--r--. 1 root root  202 Jun 29 09:15 ec2.tf
-rw-r--r--. 1 root root  643 Jun 29 09:08 main.tf
-rw-r--r--. 1 root root    0 Jun 29 09:14 terraform.tfstate           ← local file is now 0 bytes
-rw-r--r--. 1 root root 6540 Jun 29 09:14 terraform.tfstate.backup
```

All new state updates go to **S3, not locally**, so the local `terraform.tfstate` shows **zero**. *(Terraform empties it after migrating. The `.backup` is the pre-migration copy. It can be deleted once you've confirmed the S3 copy, but note it **contains secrets in plain text** like any state.)*

S3: file size grows, versioning keeps the old copy

- Before the EC2: `state/terraform.tfstate` = **6.4 KB** (09:14:27)
- After the EC2 is created: **11.6 KB** (09:18:36). The state got updated with the new instance.
- Turn on **Show versions** and both versions are there: the current **11.6 KB** and the older **6.4 KB**, each with its own Version ID.

This is the benefit of versioning. If a bad apply or a mistake corrupts the state, you can download or restore the previous version.

---

7. Lab part 4: move the state back from S3 to local

Scenario: we don't want to keep anything in the backend any more. We want everything stored **locally** again.

1. Simply **remove the backend block** (the file `back.tf`):

```
[root@Devops-server project1]# rm back.tf
rm: remove regular file 'back.tf'? y
```

2. `terraform plan` errors again:

```
Error: Backend initialization required, please run "terraform init"

Reason: Unsetting the previously set backend "s3"
```

3. Run `terraform init`, this time with the flag `-migrate-state`:

```
[root@Devops-server project1]# terraform init -migrate-state
Initializing the backend...
Terraform has detected you're unconfiguring your previously set "s3" backend.
Do you want to copy existing state to the new backend?
  Pre-existing state was found while migrating the previous "s3" backend to the
  newly configured "local" backend. No existing state was found in the newly
  configured "local" backend. Do you want to copy this state to the new "local"
  backend? Enter "yes" to copy and "no" to start with an empty state.

  Enter a value: yes
```

   After `yes`, the full state is back in the local `terraform.tfstate`.

`-migrate-state` vs `-reconfigure`

Flag	What it does	When to use
`terraform init -migrate-state`	**Copies** the existing state from the old backend to the new one (asks for confirmation)	Moving state: local → S3, S3 → local, bucket A → bucket B
`terraform init -reconfigure`	Uses the new backend config and **ignores** the old one. **No copy.**	The state is already in the new place, or you deliberately want to start fresh

---

8. Production notes (beyond the lab)

Modern S3 backend (Terraform ≥ 1.10), no DynamoDB needed:

```
terraform {
  backend "s3" {
    bucket       = "acme-tfstate-prod"
    key          = "network/vpc/terraform.tfstate"
    region       = "ap-south-1"
    encrypt      = true            # SSE on the state object
    use_lockfile = true            # S3-native locking (<key>.tflock); replaces dynamodb_table
  }
}
```

Checklist for a real state bucket:

- **Versioning ON** (rollback) + **Block Public Access** + **encryption** (SSE-S3 or SSE-KMS).
- **No `force_destroy`.** Add `lifecycle { prevent_destroy = true }`.
- **Bootstrap it separately.** Create the bucket (and table) in its own small Terraform project, or via CloudFormation or the CLI. Don't store a project's state inside the bucket that the same project manages. Our lab did exactly that, which is fine for learning, but a `destroy` there deletes the bucket holding the state.
- **One `key` per stack/environment**, e.g. `dev/app/terraform.tfstate` and `prod/app/terraform.tfstate`. Workspaces go under `env:/<workspace>/<key>` automatically.
- **IAM:** CI and engineers need `s3:GetObject/PutObject/ListBucket` on the bucket (+ `DeleteObject` for the lockfile), or `dynamodb:GetItem/PutItem/DeleteItem` on the table.
- **No variables in the `backend` block.** It's read before variables exist. For per-environment values, use **partial configuration**: `terraform init -backend-config="bucket=acme-tfstate-dev" -backend-config=dev.s3.tfbackend`.

Stuck lock? If a run crashes (Ctrl-C, CI runner killed), the lock can stay behind. First make sure **nobody is actually running**, then:

```
terraform force-unlock f740b3e6-8099-8d48-b86e-de933a3ba167    # the lock ID from the error
```

Never make `-lock=false` a habit. It switches off the very protection this chapter is about.

Command reference (this chapter)

```
terraform init                                  # first time with a backend block → offers to copy local st
terraform init -migrate-state                   # move state between backends (S3 → local, local → S3, buck
terraform init -reconfigure                     # switch backend config WITHOUT copying state
terraform init -backend-config=dev.s3.tfbackend  # partial backend config per environment
terraform state list                            # works the same with a remote backend
terraform state pull > backup.tfstate           # download the current remote state
terraform force-unlock <LOCK_ID>                # release a stale lock (only if no run is active)
terraform plan -lock-timeout=5m                 # wait up to 5 minutes for a lock instead of failing at onc
terraform plan -lock=false                      # disable locking — NOT recommended
whoami; hostname                                # matches "Who: root@Devops-server" in the lock info
aws s3api list-object-versions --bucket ramprateek --prefix state/   # see all state versions
```

Still to come

- `terraform_remote_state`: reading another stack's outputs from its S3 backend
- Backends for Azure (`azurerm`) and GCP (`gcs`), and HCP Terraform (`cloud` block)
- Restoring a previous state version from S3 after a bad apply